100 Days of Azure DevOps

Day 16 Handout — Local hooks vs server policy gates

Learning in public · Personal lab only · Educational content · Not a sales pitch

Architecture A — three layers of commit quality

Layer	Where it runs	Bypassable?	Role
Local hooks	.git/hooks/	Yes (--no-verify)	Fast developer feedback
pre-commit framework	Shared repo config	Yes if not installed	Team consistency
Branch policies / PR CI	Azure Repos server	No if enforced	Real quality lock

Architecture B — common client-side hooks

Hook	When it fires	Typical check
pre-commit	Before snapshot is written	Lint, format, block secrets
commit-msg	After message is entered	Conventional Commits pattern
pre-push	Before objects leave the machine	Tests / branch name rules

One-liner to remember

Local hooks are courtesy · Branch policy is the lock

Step-by-step lab (20–30 min)

In your local clone of **azure-100-labs**:

1. Create branch: **git switch -c feature/day16-commit-hook**
2. Add **scripts/git-hooks/commit-msg** (Conventional Commits regex)
3. Document install steps in **docs/git-hooks.md**
4. Copy hook into **.git/hooks/commit-msg**
5. Prove bad message fails; good message passes
6. Commit hook script + docs; push; open PR
7. Note: real enforcement still needs Azure Repos branch policies (Day 19)

commit-msg check (lab)

```
#!/bin/sh
msg=$(head -n1 "$1")
echo "$msg" | grep -qE '^(feat|fix|docs|chore|refactor|test|ci)(\(.*\))?: .+' || {
  echo "Use: feat: short description"; exit 1
}
```

Done checklist

- I can explain local hooks vs server branch policies
- commit-msg hook script committed under scripts/git-hooks/
- Bad commit message rejected locally

- Understood --no-verify is not a production control

Tomorrow — Day 17

Fork workflows & permissions — who can push what, and why least privilege matters.

#100DaysOfAzureDevOps #Azure #DevOps #CloudComputing #LearningInPublic